

MARMARA UNIVERSITY

FACULTY OF ENGINEERING



EE4077

FUNDAMENTALS OF MACHINE LEARNING

HOMEWORK VI
SOLUTIONS

SOLUTION FOR QUESTION 1

a)

The log-likelihood can be calculated as follows

$$L(x|\theta) = \ln\left(\prod_{i=1}^{n} f(x_i|\theta)\right) = \ln\left(\prod_{i=1}^{n} \lambda e^{-\lambda x_i}\right) = \sum_{i=1}^{n} \ln\left(\lambda e^{-\lambda x_i}\right)$$

$$L(x|\theta) = \sum_{i=1}^{n} \ln(\lambda) + \ln\left(e^{-\lambda x_i}\right)$$

$$L(x|\theta) = \sum_{i=1}^{n} \ln(\lambda) + (-\lambda x_i)\ \ln(e)$$

$$L(x|\theta) = \mathrm{n}\ln(\lambda) - \lambda \sum_{i=1}^{n} (x_i)$$

b)

Conditional expectation of X_i when assuming observe $Y_i = y_i < c_i$

$$E[X_i|Y_i = y_i < c_i] = E[X_i|X_i < c_i]$$

$$E[X_i|X_i < c_i] = \frac{\int_0^{c_i} x_i \lambda e^{-\lambda x_i}}{P(X_i < c_i)} = \frac{-c_i e^{-\lambda c_i} + \left[\frac{1 - e^{-\lambda c_i}}{\lambda}\right]}{1 - e^{-\lambda c_i}}$$

$$E[X_i|X_i < c_i] = \frac{1}{\lambda} - \frac{c_i e^{-\lambda c_i}}{1 - e^{-\lambda c_i}}$$

c)

From memorylessness property of exponential distribution

$$Y_i = c_i \rightarrow X_i \geq c_i$$

$$E[X_i|X_i \geq c_i] = \frac{\int_0^{c_i} x_i \lambda e^{-\lambda x_i}}{P(X_i \geq c_i)} = \frac{-c_i e^{-\lambda c_i} + \left[\frac{e^{-\lambda c_i}}{\lambda}\right]}{e^{-\lambda c_i}}$$

$$E[X_i|X_i \geq c_i] = c_i + \frac{1}{\lambda}$$

d)

E step can be driven as follows

$$Q(\lambda|\lambda^t) = E[L|y, \lambda^t]$$

$$Q(\lambda|\lambda^t) = nln(\lambda) - \lambda \sum_{i=1}^{n} E[X_i|Y_i, \lambda^t]$$

$$Q(\lambda|\lambda^t) = nln(\lambda) - \lambda \sum_{i=1}^{n} I_{X_i<c_i} E[X_i|X_i < c_i] + I_{X_i \geq c_i} E[X_i|X_i \geq c_i]$$

$$Q(\lambda|\lambda^t) = nln(\lambda) - \lambda \sum_{i=1}^{n} \frac{1}{\lambda} + \frac{c_i e^{-\lambda c_i}}{1 - e^{-\lambda c_i}} + c_i + \frac{1}{\lambda}$$

$$Q(\lambda|\lambda^t) = nln(\lambda) - \lambda \sum_{i=1}^{n} \frac{2}{\lambda} + \frac{c_i e^{-\lambda c_i}}{1 - e^{-\lambda c_i}} + c_i$$

e)

M step can be calculated by maximizing the E step

$$\underset{\lambda}{\operatorname{argmax}}\left(Q(\lambda|\lambda^t)\right) = \underset{\lambda}{\operatorname{argmax}}\left(nln(\lambda) - \lambda \sum_{i=1}^{n} \frac{2}{\lambda} + \frac{c_i e^{-\lambda c_i}}{1 - e^{-\lambda c_i}} + c_i\right)$$

SOLUTION FOR QUESTION 2

a)

After centering the X, we have following input features

$$X = \begin{bmatrix} -1 & -1 & -2 \\ 0 & 1 & 1 \\ 1 & 0 & 1 \end{bmatrix}$$

With following python code, covariance matrix and corresponding eigenvalues and eigenvectors can be calculated

```
X = np.array([[-1, -1, -2], [0, 1, 1], [1, 0, 1]])

C_X = np.matmul(np.transpose(X), X)
eigenvalues, eigenvectors = np.linalg.eig(C_X)
```

From code above, covariance matrix and eigenvalues and eigenvectors are shown as follows

$$C_x = \begin{bmatrix} 2 & 1 & 3 \\ 1 & 2 & 3 \\ 3 & 3 & 6 \end{bmatrix}$$

$$Eigenvalues: [9.00000000e+00\ 1.00000000e+00\ 3.05893206e-16]$$

$$Eigenvectors:$$

$$[[-4.08248290e-01\ -7.07106781e-01\ -5.77350269e-01]$$

$$[-4.08248290e-01\ 7.07106781e-01\ -5.77350269e-01]$$

$$[-8.16496581e-01\ 1.88941791e-16\ 5.77350269e-01]]$$

b)

Two principle components can then be computed as first two eigenvectors since eigenvalues of 9 and 1 I the greatest. Projection matrix is shown below

$$P = \begin{bmatrix} -0.408 & -0.408 \\ -0.707 & -0.707 \\ -0.577 & -0.577 \end{bmatrix}$$

c)

With following python code, each data can be projected on to 2-D subspace.

```
x1 = np.array([0, -1, -2])
x2 = np.array([1, 1, 1])
x3 = np.array([2, 0, 1])

x_mean = np.mean([x1, x2, x3], axis=0)

a11 = np.dot(x1 - x_mean, principal_components[:, 0])
a12 = np.dot(x1 - x_mean, principal_components[:, 1])
b3 = np.dot(x2 - x_mean, np.cross(principal_components[:, 0],
principal_components[:, 1]))

a21 = np.dot(x2 - x_mean, principal_components[:, 0])
a22 = np.dot(x2 - x_mean, principal_components[:, 1])

a31 = np.dot(x3 - x_mean, principal_components[:, 0])
a32 = np.dot(x3 - x_mean, principal_components[:, 1])

x1_prime = a11 * principal_components[:, 0] + a12 * principal_components[:,
1] + b3 * np.cross(principal_components[:, 0], principal_components[:, 1])
x2_prime = a21 * principal_components[:, 0] + a22 * principal_components[:,
1] + b3 * np.cross(principal_components[:, 0], principal_components[:, 1])
x3_prime = a31 * principal_components[:, 0] + a32 * principal_components[:,
1] + b3 * np.cross(principal_components[:, 0], principal_components[:, 1])

diff_x1 = np.linalg.norm(x1 - x1_prime)
diff_x2 = np.linalg.norm(x2 - x2_prime)
diff_x3 = np.linalg.norm(x3 - x3_prime)
```

The obtained results are shown below

a_{11}: 2.449489742783178

a_{12}: $-$ 4.481667407267538$e-17$

a_{21}: $-$ 1.2247448713915887

a_{22}: 0.7071067811865476

a_{31}: $-$ 1.2247448713915894

a_{32}: $-$ 0.7071067811865475

b_3: 1.1102230246251565$e-16$

$x1'_{projected}$: [-1 -1 -2]

$||x1' - x1||$: 1.0000000000000007

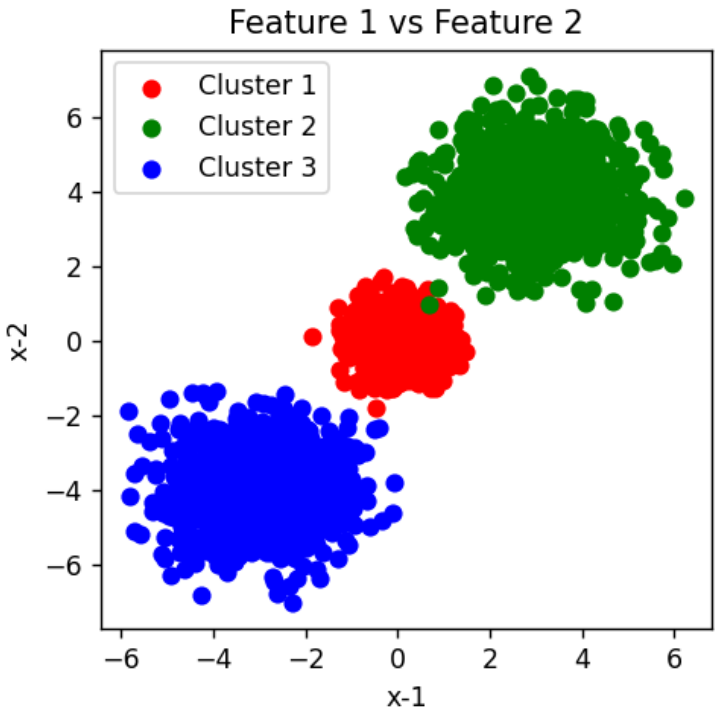
$x2'_{projected}$: [1.75121059$e-16$ 1.00000000$e+00$ 1.00000000$e+00$]

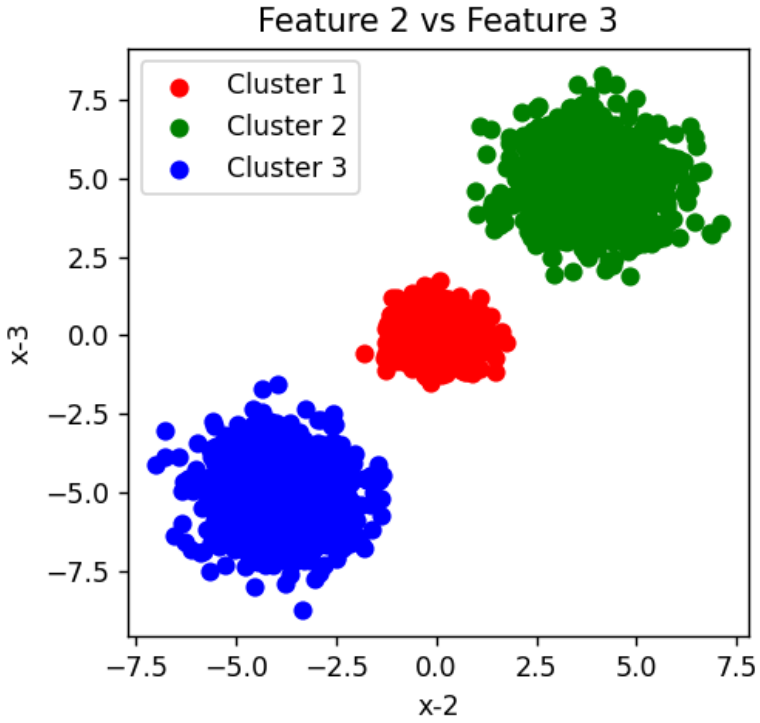
$||x2' - x2||$: 0.9999999999999998

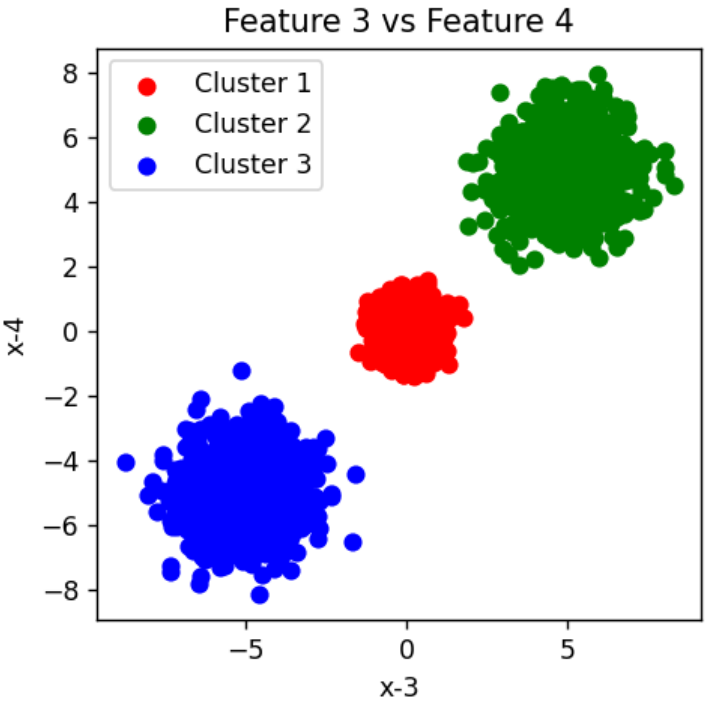
$x3'_{projected}$: [1.00000000$e+00$ 1.19609907$e-16$ 1.00000000$e+00$]

$||x3' - x3||$: 0.9999999999999993

SOLUTION FOR QUESTION 3







The code for this question is shared in the Appendix section of this PDF.

APPENDIX

```
import numpy as np
from scipy.stats import multivariate_normal
from sklearn.mixture import GaussianMixture
import matplotlib.pyplot as plt


# Read the data from the text file
with open('gmm_data.txt', 'r') as file:
    data = []
    for line in file:
        row = [float(value) for value in line.split()]
        data.append(row)

data = np.array(data)
num_components = 3
num_samples, num_features = data.shape

def EM_func(data, num_components, means=np.random.randn(num_components,
num_features),
            covariances = [np.eye(num_features) for _ in
range(num_components)],
            weights = np.ones(num_components) / num_components,
            num_iterations=100, tol=1e-4):

    num_samples, _ = data.shape

    for iteration in range(num_iterations):
        # Expectation step
        r = np.zeros((num_samples, num_components))
        for c in range(num_components):
            r[:, c] = weights[c] * multivariate_normal.pdf(data, means[c],
covariances[c])

        r /= r.sum(axis=1, keepdims=True)

        # Maximization step
        N_c = r.sum(axis=0)
        means = np.dot(r.T, data) / N_c[:, np.newaxis]
        covariances = [np.dot((r[:, c] * (data - means[c]).T), (data -
means[c])) / N_c[c]
                       for c in range(num_components)]
        weights = N_c / num_samples

        # Calculate log-likelihood to check convergence
        log_likelihood = np.sum(np.log(np.sum(weights[c] *
multivariate_normal.pdf(data, means[c], covariances[c])[:, np.newaxis],
axis=0)))

        # Check for convergence
        if iteration > 0 and np.abs(log_likelihood - prev_log_likelihood) <
tol:
            break

        prev_log_likelihood = log_likelihood

    return means, covariances, weights
```

```
# Part a
gmm = GaussianMixture(n_components=num_components, random_state=0,
tol=0.001)
gmm.fit(data)

#Part b
colors = ['red', 'green', 'blue']
cluster_assignments = gmm.predict(data)

plt.figure(figsize=(12, 4))
for dim in range(3):
    plt.subplot(1, 3, dim + 1)
    for i in range(num_components):
        cluster_points = data[cluster_assignments == i]
        plt.scatter(cluster_points[:, dim], cluster_points[:, dim + 1],
color=colors[i], label=f'Cluster {i+1}')
    plt.title(f'Dimension {dim+1} vs Dimension {dim+2}')
    plt.xlabel(f'x-{dim+1}')
    plt.ylabel(f'x-{dim+2}')
    plt.legend()

plt.tight_layout()

#Part c
custom_means_c, _, _ = EM_func(data, num_components, weights=gmm.weights_,
covariances=gmm.covariances_)


#Part d
custom_means_d, _, custom_weights_d = EM_func(data, num_components,
covariances=gmm.covariances_)

# Print means and covariances of the custom model
print("\nCustom GMM:")
print(custom_means_c)

# Print means and covariances of the custom model
print("\nCustom GMM:")
print(custom_weights_d)

# Print means and covariances of the scikit-learn model
print("\nScikit-learn GMM:")
print(gmm.means_)

plt.show()
```